

ORDER API

Backoffice et API REST headless de gestion e-commerce sécurisé

Documentation du projet - Planification des Sprints

Version 0.2.0 - 2026

1. Définition du problème

Quand une PME lance son e-commerce, tout va bien au début.

Les outils sont simples, rapides à mettre en place, efficaces pour démarrer.

Mais très vite, dès que l'activité augmente, ces mêmes outils deviennent des freins.

Les entreprises perdent de la flexibilité, deviennent dépendantes techniquement, et surtout... elles n'ont plus une vision claire de leurs données.

Le vrai problème n'est pas technologique, il est décisionnel.

2. Solution

Concrètement, ORDER API fonctionne comme une brique backend API-first.

Toute la logique métier est centralisée : commandes, statuts, sécurité, événements.

Le contrôle d'accès par rôle garantit que chaque utilisateur voit uniquement ce qui le concerne.

On n'a pas cherché à être exhaustif, mais à être fiable, sécurisé et observable.

3. Vision du produit

Plutôt que d'ajouter une couche de complexité, on a pris le problème à l'envers.

ORDER API est né d'une question simple : comment redonner aux PME le contrôle, sans leur imposer un nouvel outil rigide ?

La réponse, c'est une approche headless :

- Un backend indépendant
- Un frontend libre
- Et des données pleinement maîtrisées

Objectifs de la vision

- La plateforme est conçue pour répondre aux besoins des administrateurs, commerciaux et opérationnels, en leur donnant un accès direct et adapté aux données métier.
- Permettre aux administrateurs un accès en libre-service aux exports de données en CSV et Excel.
- Fournir des tableaux de bord métier en temps réel.
- Automatiser les rapports KPI périodiques via des scripts Python planifiés.
- Mettre en place un pipeline de données automatisé (collecte, transformation, stockage).

Valeur ajoutée

- Meilleure gouvernance des données : les clients ne peuvent pas accéder aux données d'autres utilisateurs
- Efficacité opérationnelle : les administrateurs exportent les données sans accès direct à la base de données
- Visibilité métier : les dashboards Grafana mettent en avant des indicateurs actionnables
- Architecture scalable : le système RBAC est prêt à accueillir des rôles supplémentaires

Au-delà de l'API transactionnelle, ORDER API pose les bases d'une couche analytique : collecte automatisée des KPI, exports structurés et dashboards métier. C'est le premier maillon d'un pipeline de données orienté décision.

4. Rôles et permissions

Rôle	Profil	Permissions	Implémenté
User	client	Passer des commandes, consulter ses propres commandes et factures	v0.2.0
Admin	administrateur	Gérer le catalogue, exporter les données, accéder aux dashboards, superviser l'ensemble de l'activité	v0.2.0
Commercial	équipe métier	Consulter les commandes, suivre les paiements, analyser la performance commerciale	Roadmap
Opérationnel	équipe métier / exploitation	Superviser les métriques, surveiller les incidents, accéder aux dashboards temps réel	Roadmap

Les rôles Commercial et Opérationnel sont des déclinaisons prévues du rôle admin. Ils seront implémentés dans une version future avec des permissions granulaires dédiées

5. Product Backlog

Dans ce backlog, deux types de user stories sont distingués :

- User stories métier :
 - Exprimées du point de vue des utilisateurs finaux (administrateurs, commerciaux, opérationnels).
 - Elles décrivent des besoins fonctionnels directement liés à l'usage du produit et à la création de valeur métier.
- User stories techniques :
 - Exprimées du point de vue du système ou des développeurs.
 - Elles concernent des besoins d'architecture, de sécurité, de performance ou de maintenabilité.
 - Elles sont indispensables au bon fonctionnement du produit mais ne sont pas directement visibles par les utilisateurs.

Cette distinction permet de mieux structurer le développement et de prioriser la valeur métier tout en garantissant la robustesse technique.

EPIC 1 - Contrôle d'accès basé sur les rôles (RBAC)

ID	User Story	Type	Rôle	Priorité	Story Points	Status
US1	En tant qu'administrateur, je veux que les routes de création/modification/suppression des produits soient protégées afin que seuls les admins puissent modifier le catalogue	Métier	admin	Haute	5	À faire
US2	En tant que client, je veux accéder uniquement à mes propres commandes et factures afin que mes données restent privées	Métier	user	Haute	3	À faire
US3	En tant qu'administrateur, je veux consulter toutes les commandes et tous les utilisateurs afin d'avoir une visibilité opérationnelle complète	Métier	admin	Haute	3	À faire
US4	En tant que système, je veux centraliser la gestion des rôles afin de garantir un contrôle d'accès cohérent sur toutes les routes	Technique	système	Haute	5	À faire

EPIC 2 - Exports de données (CSV / Excel)

ID	User Story	Type	Rôle	Priorité	Story Points	Status
US5	En tant qu'administrateur, je veux exporter toutes les commandes en CSV afin de les analyser dans Excel ou Python	Métier	admin	Haute	5	À faire
US6	En tant qu'administrateur, je veux exporter tous les produits en CSV afin d'auditer le catalogue	Métier	admin	Moyenne	3	À faire
US7	En tant qu'administrateur, je veux exporter tous les utilisateurs en CSV afin de gérer la base clients	Métier	admin	Moyenne	3	À faire
US8	En tant qu'administrateur, je veux exporter toutes les factures en Excel (.xlsx) afin de les utiliser en comptabilité	Métier	admin	Moyenne	5	À faire
US9	En tant qu'administrateur, je veux filtrer les exports par plage de dates afin de cibler mon analyse	Métier	admin	Faible	2	À faire

EPIC 3 - Tableaux de bord métier (Grafana)

ID	User Story	Type	Rôle	Priorité	Story Points	Status
US10	En tant qu'administrateur, je veux visualiser les commandes par jour afin de suivre le volume métier	Métier	admin	Haute	3	À faire
US11	En tant qu'administrateur, je veux visualiser le temps de traitement moyen des commandes afin de détecter les ralentissements	Métier	admin	Haute	5	À faire
US12	En tant qu'administrateur, je veux visualiser des alertes quand le taux d'erreur dépasse 5% afin d'être notifié des incidents	Métier	admin	Moyenne	5	À faire
US13	En tant qu'administrateur, je veux consulter le chiffre d'affaires par jour afin de suivre la performance financière	Métier	admin	Faible	3	À faire

EPIC 4 - Pipeline de données & automatisation

Objectif

Construire un pipeline de données automatisé permettant de calculer, stocker et exposer des KPI métier.

ID	User Story	Type	Rôle	Priorité	Story Points	Status
US14	En tant qu'administrateur, je veux un script Python qui calcule les KPI quotidiens (nombre de commandes, délai moyen, taux d'erreur) afin d'obtenir un résumé périodique	Métier (orienté data)	admin	Haute	8	À faire
US15	En tant qu'administrateur, je veux que le script KPI s'exécute automatiquement chaque jour afin que le reporting ne nécessite aucune action manuelle	Technique	admin	Haute	5	À faire
US16	En tant qu'administrateur, je veux que le rapport KPI soit exporté en CSV afin de pouvoir le stocker et l'analyser dans le temps	Métier	admin	Moyenne	3	À faire
US17	En tant que système, je veux stocker les KPI calculés dans une table dédiée afin de conserver un historique analytique interrogeable	Technique	système	Moyenne	5	À faire

6. Critères d'acceptation

Objectif des critères

Les critères d'acceptation définissent les conditions de validation des User Stories.

Ils permettent de s'assurer que les fonctionnalités répondent aux besoins métier et sont testables.

US1 - Protection des routes produits (RBAC admin)

ID	Scénario	Given (Étant donné)	When (Quand)	Then (Alors)
AC1	Création refusée sans rôle admin	Je suis authentifié avec le rôle user	J'envoie POST /products	Je reçois une réponse 403 Forbidden
AC2	Modification refusée sans rôle admin	Je suis authentifié avec le rôle user	J'envoie PATCH /products/{id}	Je reçois une réponse 403 Forbidden
AC3	Suppression refusée sans rôle admin	Je suis authentifié avec le rôle user	J'envoie DELETE /products/{id}	Je reçois une réponse 403 Forbidden
AC4	Création autorisée pour l'admin	Je suis authentifié avec le rôle admin	J'envoie POST /products avec un body valide	Le produit est créé et je reçois 201 avec les données du produit
AC5	Accès non authentifié refusé	Je ne suis pas authentifié	J'envoie POST /products	Je reçois une réponse 401 Unauthorized
AC6	Consultation publique autorisée	Je ne suis pas authentifié	J'envoie GET /products	Je reçois 200 avec la liste des produits

US5 - Export des commandes en CSV

ID	Scénario	Given (Étant donné)	When (Quand)	Then (Alors)
AC1	Export refusé sans rôle admin	Je suis authentifié avec le rôle user	J'envoie GET /exports/orders	Je reçois une réponse 403 Forbidden
AC2	Export refusé sans authentification	Je ne suis pas authentifié	J'envoie GET /exports/orders	Je reçois une réponse 401 Unauthorized
AC3	Export CSV généré pour l'admin	Je suis authentifié avec le rôle admin	J'envoie GET /exports/orders	Je reçois un fichier CSV téléchargeable avec l'en-tête Content-Type: text/csv
AC4	Contenu du fichier correct	Des commandes existent en base	Le fichier CSV est généré	Il contient les colonnes id, user_id, status, total_cents, currency, created_at
AC5	Fichier vide si aucune commande	Aucune commande n'existe en base	J'envoie GET /exports/orders	Je reçois un fichier CSV avec uniquement la ligne d'en-tête
AC6	Filtre par date fonctionnel	Je suis administrateur et des commandes existent	J'envoie GET /exports/orders?from_date=2026-01-01&to_date=2026-01-31	Le fichier ne contient que les commandes de la période demandée

US10 - Visualisation des commandes par jour

ID	Scénario	Given (Étant donné)	When (Quand)	Then (Alors)
AC1	Affichage du graphique	Je suis un administrateur authentifié	J'accède au dashboard Grafana	Je vois un graphique affichant le nombre de commandes par jour
AC2	Exactitude des données	Il existe des commandes en base de données	Le graphique est affiché	Les données correspondent aux commandes enregistrées
AC3	Filtrage par date	Je sélectionne une plage de dates	Le Dashboard est mis à jour	Seules les commandes de la période sélectionnée sont affichées
AC4	Sécurité d'accès	Je ne suis pas administrateur	J'essaie d'accéder au Dashboard	L'accès est refusé
AC5	Temps réel	Les données sont collectées	Le Dashboard est affiché	Les données sont mises à jour en quasi-temps réel (< 1 min)

US14 - Script KPI quotidien

ID	Scénario	Given (Étant donné)	When (Quand)	Then (Alors)
AC1	Calcul du nombre de commandes	Des commandes existent en base pour la journée	Le script kpi_report.py s'exécute	Le rapport contient le nombre exact de commandes du jour
AC2	Calcul du délai moyen	Des commandes payées existent avec des timestamps	Le script s'exécute	Le rapport contient le délai moyen en secondes entre création et paiement
AC3	Calcul du taux d'erreur	Des requêtes HTTP ont été enregistrées	Le script s'exécute	Le rapport contient le pourcentage de réponses 5xx sur le total
AC4	Export CSV généré	Le script s'exécute sans erreur	L'exécution se termine	Un fichier kpi_YYYY-MM-DD.csv est créé dans le dossier reports/
AC5	Exécution automatique à 06h00	APScheduler est démarré avec l'API	Il est 06h00	Le script s'exécute automatiquement sans intervention manuelle
AC6	Idempotence du script	Le script a déjà été exécuté aujourd'hui	Le script s'exécute une deuxième fois	Le fichier CSV existant est écrasé sans erreur

7. Planification des Sprints (3 semaines)

7.1. Lecture produit de la roadmap

Chaque sprint correspond à une étape de maturité produit. Les choix techniques sont guidés par la valeur métier délivrée aux équipes (autonomie, visibilité, décision), et non par une logique d'implémentation isolée.

7.2. SPRINT 1 (Semaine 1) : RBAC et contrôle d'accès

Objectif produit

Sécuriser les accès et établir une gouvernance claire des données

Objectif du sprint

Appliquer le contrôle d'accès basé sur les rôles sur toutes les routes existantes. Les utilisateurs n'accèdent qu'à leurs propres données ; les administrateurs ont un accès complet.

User Stories

- US1 : Protéger les routes d'écriture des produits (POST/PUT/PATCH/DELETE) avec `require_admin`
- US2 : Filtrer les commandes et factures par utilisateur authentifié (rôle user)
- US3 : Permettre aux administrateurs de lister toutes les commandes et tous les utilisateurs
- US4 : Implémenter une dépendance `require_role()` réutilisable

Tâches techniques

ID	Tâche	US liée	Fichier(s)	Est.
T1	Créer la fonction factory require_role(role)	US4	app/core/auth.py	1h
T2	Protéger POST/PUT/DELETE /products avec require_admin	US1	app/api/products.py	1h
T3	Filtrer GET /orders pour ne retourner que les commandes de l'utilisateur connecté	US2	app/api/orders.py	2h
T4	Filtrer GET /invoices pour ne retourner que les factures de l'utilisateur connecté	US2	app/api/invoices.py	1h
T5	Ajouter GET /admin/orders (toutes les commandes) et GET /admin/users (tous les utilisateurs)	US3	app/api/admin.py (nouveau)	2h
T6	Écrire les tests pour chaque rôle (accès user, accès admin, accès interdit)	US1, 2, 3	tests/test_rbac.py (nouveau)	3h

Livrables

- Dépendance require_role() fonctionnelle et testée
- Toutes les routes protégées selon le tableau des rôles
- Nouveau fichier de tests : tests/test_rbac.py
- Tous les tests existants toujours verts (CI green)

7.3. SPRINT 2 (Semaine 2) : Exports de données (CSV / Excel)

Objectif produit

Rendre les équipes métier autonomes dans l'exploitation de données.

Objectif du sprint

Exposer des endpoints d'export sécurisés pour les admins..

User Stories

- US5 : Export des commandes en CSV
- US6 : Export des produits en CSV
- US7 : Export des utilisateurs en CSV
- US8 : Export des factures en Excel (.xlsx)
- US9 : Filtre optionnel par plage de dates sur les exports

Tâches techniques

ID	Tâche	US liée	Fichier(s)	Est.
T7	Ajouter openpyxl dans requirements.txt	US8	requirements.txt	15min
T8	Créer app/api/exports.py avec GET /exports/orders (CSV)	US5	app/api/exports.py (nouveau)	2h
T9	Ajouter GET /exports/products et GET /exports/users (CSV)	US6, 7	app/api/exports.py	1h
T10	Ajouter GET /exports/invoices (.xlsx avec openpyxl)	US8	app/api/exports.py	2h
T11	Ajouter les paramètres optionnels ?from_date=&to_date= sur l'export des commandes (filtre dates)	US9	app/api/exports.py	1h
T12	Enregistrer le router exports dans app/main.py	US5, 6, 7, 8, 9	app/main.py	15min
T13	Écrire les tests pour les routes d'export (auth, format, contenu)	US5, 6, 7, 8, 9	tests/test_exports.py (nouveau)	2h

Livrables

- 4 endpoints d'export actifs et protégés par require_admin
- Exports CSV téléchargeables depuis GET /exports/orders, /products, /users
- Export Excel téléchargeable depuis GET /exports/invoices
- Nouveau fichier de tests : tests/test_exports.py

7.4. SPRINT 3 (Semaine 3) - Dashboards Grafana

Objectif produit

Permettre le pilotage métier en temps réel.

Objectif du sprint

Mettre en place les dashboards métiers pour le suivi des performances.

User Stories

- US10 : Commandes par jour
- US11 : Temps de traitement moyen
- US12 : Alertes taux d'erreur
- US13 : Chiffre d'affaires par jour

Tâches techniques

ID	Tâche	US liée	Fichier(s)	Est.
T14	Ajouter des métriques Prometheus (orders_total, order_processing_seconds)	US10, 11, 12	app/core/metrics/prometheus.py	2h
T15	Créer un panel Grafana : commandes/jour (bar chart)	US10	monitoring/grafana/dashboards/order_api.json	1h
T16	Créer un panel Grafana temps réel de traitement (jauge)	US11	monitoring/grafana/dashboards/order_api.json	1h
T17	Configurer alerte taux d'erreur	US12	monitoring/grafana/dashboards/order_api.json	1h

Livrables

- 3 nouveaux panels Grafana (commandes/jour, temps de traitement moyen, alerte taux d'erreur)
- Tous les tests existants toujours verts — CI green

7.5. SPRINT 4 (Semaine 4) – Automatisation KPI

Objectif produit

Industrialiser la production et la diffusion des KPI

Objectif du sprint

Automatiser la génération et l'export des indicateurs métiers.

User Stories

- US14 : Calcul des KPI
- US15 : Exécution automatique
- US16 : Export des KPI

Tâches techniques

ID	Tâche	US liée	Fichier(s)	Est.
T18	Écrire scripts/kpi_report.py : récupérer les données depuis la DB, calculer les KPI, exporter en CSV	US14	scripts/kpi_report.py (nouveau)	3h
T19	Intégrer APScheduler dans FastAPI pour exécuter kpi_report.py tous les jours à 06h00	US15	app/main.py ou app/core/scheduler.py	2h
T20	Ajouter apscheduler dans requirements.txt et tester le job planifié	US15, 16	requirements.txt, tests/test_scheduler.py	1h

Livrables

- scripts/kpi_report.py produisant un CSV quotidien
- APScheduler exécutant le script automatiquement à 06h00
- Tous les tests verts - CI green

8. Architecture technique

Couche	Technologie	Rôle
API	FastAPI 0.116	Endpoints REST, authentification JWT, middleware RBAC
Base de données	PostgreSQL 15	Commandes, produits, utilisateurs, factures
Cache / Rate limiting	Redis 7	Liste noire JWT, rate limiting, idempotence
Métriques	prometheus-client 0.22	Exposition de /metrics pour Prometheus
Monitoring	Prometheus + Grafana 11.4	Collecte des métriques et dashboards
Exports	openpyxl + csv (stdlib)	Génération de fichiers CSV et Excel
Automatisation	APScheduler	Jobs KPI planifiés intégrés dans FastAPI
Intégration continue	GitHub Actions	Suite de tests automatisée à chaque push
Pipeline de données	Pipeline ETL léger (Extract-Transform-Load)	Collecte, transformation et agrégation des données métier
Stockage analytique	PostgreSQL (schéma analytics ou tables dédiées)	Stockage historique des KPI (revenu, commandes, erreurs)
Vers une séparation OLTP / OLAP (simplifiée)	Base transactionnelle isolée des requêtes analytiques (roadmap)"	Optimisation des requêtes de reporting et dashboards

9. État actuel (base v0.2.0)

Fonctionnalité	État	Notes
Auth (register, login, logout, refresh)	Fait	Tokens JWT accès + refresh
users/me	Fait	Retourne le profil de l'utilisateur connecté
RBAC require_role () + filtrage ownership	Fait	require_role(), orders et invoices filtrés par user
CRUD Produits	Fait	GET/POST/PUT/DELETE
Commandes (création, articles, paiement, statut)	Fait	Filtrées par utilisateurs authentifiés (ownership)
Factures	Fait	Créées automatiquement après paiement
Paiement	Fait	Idempotence_Key (Redis)
Rate limiting (Redis)	Fait	Par IP et par famille de routes
Middleware d'audit	Fait	Écrit audit_log.csv
Prometheus /metrics	Fait	http_requests_total + histogramme de durée
Dashboard Grafana	Fait	Latency p95, RPS, taux d'erreur
Tests	Faits	25 tests pytest (auth, orders, products, invoices)
Routes admin (GET /admin/orders, /admin/users)	En cours	T5 — Sprint 1 non terminé
Exports de données	Non commencé	Sprint 2
Tableaux de bord métier	Non commencé	Sprint 3
Automatisation / scheduler	Non commencé	Sprint 4
Pipeline de données (KPI, exports, dashboards)	Non commencé	Structuration prévue — Sprint 2 à 4